



عملي

4



9



1080 sp

كلية الهندسة المعلوماتية

السنة الثالثة

تقنيات التحكم في الاستنتاج وإدارة قاعدة

عملي مشترك



RB Informatcs; 10/05/2026

محتوى مجاني غير مخصص للبيع التجاري

مبادئ الذكاء الصناعي

تحدثنا في المحاضرات السابقة عن علاقة السلف (Ancestor) وتعلمنا انها تعتمد على قاعدتين أساسيتين هي:

- القاعدة الأولى (الأبوة المباشرة): يكون الشخص X سلفاً لـ Y إذا كان X هو الأب المباشر (Parent) لـ Y.
- القاعدة الثانية (العلاقة المتعدية/العودية): يكون الشخص X سلفاً لـ Y إذا وجد شخص وسيط (Z)، بحيث يكون X هو السلف لـ Z، وهذا الوسيط Z هو الأب المباشر لـ Y.

مثال:

ليكن لدينا أحمد A، أبو أحمد: هو والد أحمد AA، أبو أمجد: هو جد أحمد AAA

العلاقات المعرفة في قاعدة المعرفة هي علاقات أبوة مباشرة كالتالي:

`parent(abu_ahmad, ahmad), parent(abu_amjad, abu_ahmad)`

والآن سنتتبع الاستعلام: عندما نسأل هل أبو أمجد هو سلف أحمد بالاستدعاء هذا: `ancestor(abu_amjad, ahmad)`

1. تجربة القاعدة الأولى: يحاول البرنامج مطابقة الاستعلام مع قاعدة الأبوة المباشرة: هل أبو أمجد هو الأب المباشر لأحمد؟
2. النتيجة هي فشل (Fail) لأن الجد ليس الأب المباشر.
3. وتتم عملية التراجع: (Backtracking) يعود المحرك للخلف ليبحث عن خيار آخر، فينتقل إلى القاعدة الثانية. تحليل القاعدة الثانية: يبحث المحرك عن شخص وسيط (Z) يحقق شرطين:
 - أن يكون هذا الوسيط (Z) هو ابن (أو ينحدر من سلف) أي: أبو أمجد
 - أن يكون هذا الوسيط (Z) هو الأب المباشر لأحمد
4. إيجاد الوسيط: يجد المحرك أن الشخص الذي هو أب مباشر لأحمد هو أبو أحمد أي أن `Z = abu_ahmad`
5. العودية (Recursion): الآن ينتقل المحرك للتأكد من الجزء الأول: هل أبو أمجد هو سلف لأبو أحمد؟
6. يعود المحرك مرة أخرى للقاعدة الأولى ليجد أن أبو أمجد هو فعلاً الأب المباشر لـ أبو أحمد هنا تتحقق النتيجة ويظهر نجاح (Success)

ملاحظة: أن محرك الاستنتاج لا يكتفي بإيجاد حل واحد؛ فبعد أن يكتشف أن الجد سلف للحفيد، قد يستمر في البحث وتجربة قواعد أخرى أو مسارات بديلة (مثل البحث عن أبناء آخرين) مما يستهلك وقتاً طويلاً دون فائدة

تقنيات التحكم في الاستنتاج وإدارة قاعدة المعرفة:

:(The Cut - !)

تعد هذه الأداة التي يرمز لها بعلامة التعجب بمثابة إشارة توقف تخبر محرك الاستنتاج في برولوج بأنه لا داعي للعودة وتجربة قواعد أخرى بمجرد الوصول لهذه النقطة وتحقق الشروط التي تسبقها وهذا يساعد في تحسين الأداء وتخفيف تعقيد الكود بشكل كبير وتفاذي عمليات البحث الزائدة بالعودة لمثال علاقة السلف المشكلة كانت أن المحرك لا يكتفي بإيجاد رابط واحد بل يستمر في تجربة كافة القواعد المتاحة فإذا كان لدينا الجد أبو امجد والأب أبو أحمد والابن أحمد وسألنا هل الجد سلف للحفيد سيقوم البرنامج بفحص قاعدة الأبوة المباشرة أولاً ثم يتراجع لبحث في القاعدة الثانية وهكذا لذلك سنستفيد من هذه الأداة حيث بمجرد إيجاد الرابط سنضع الكت لنخبره أن يكتفي بهذا الحل ولا يكمل البحث في فروع أخرى لشجرة العائلة

المثال باستخدام ال Cut:

```
parent(abu_amjad,abu_ahmad).
parent(abu_ahmad,ahmad).
ancestor(X,Y) :- parent(X,Y),!.
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

○ شرح كيف سيعمل الاستعلام: ancestor(abu_amjad, ahmad)

1. المحاولة الأولى: سيبحث البرولوج في القاعدة الأولى: هل parent(abu_amjad, ahmad) النتيجة فشل، لذا لن يصل لل Cut وسينتقل للقاعدة الثانية.
2. المحاولة الثانية: في القاعدة الثانية، سيبحث عن وسيط Z بحيث يكون parent(abu_amjad, Z). أن Z = abu_ahmad.
3. الاستدعاء الذاتي: سيقوم الآن باستدعاء ancestor(abu_ahmad, ahmad).
4. العودة للقاعدة الأولى: سيفحص parent(abu_ahmad, ahmad).
5. النتيجة ستكون True



6. بمجرد تحقق الشرط سيجد Cut (!) هذا يعني أن البرولوج سيثبت هذه النتيجة ويغلق مسار البحث لهذا الاستدعاء ولن يحاول تجربة القاعدة الثانية بحثاً عن سلف للسلف لهذا الفرع.

مثال 2:

تعتمد الدالة على قاعدتين لفحص الرقمين وإعادة الأكبر بينهما في المتغير الثالث

- القاعدة الأولى. $X \geq Y$:- $\max(X, Y, X)$
- تعيد X إذا كان أكبر من أو يساوي Y
- القاعدة الثانية. $Y > X$:- $\max(X, Y, Y)$
- تعيد Y إذا كان أكبر من X
- إذا فرضنا هذا الاستدعاء $\max(3, 2, Z)$ سيجد البرولوج الحل في القاعدة الأولى لأن X فعلاً هي أكبر من Y ولكنه سيعود ليحرب القاعدة الثانية أيضاً (Backtracking)، وهذا استهلاك غير ضروري للموارد

الحل باستخدام الـ Cut:

نضيف إشارة الكت لإيقاف عملية التراجع بمجرد تحقق الشرط الأول:

$\max(X, Y, X) :- X \geq Y, !.$

$\max(X, Y, Y).$

هنا، إذا كان $X \geq Y$ وتحققت Cut فلن ينظر البرولوج إلى القاعدة الثانية أبداً

ولكن المشكلة أن الطريقة السابقة قد تخطئ عند إجراء استعلام مثل $\max(3, 2, 2)$.

تتبع الاستعلام:

المحرك يحاول مطابقة القاعدة الأولى $\max(X, Y, X)$ مع $\max(3, 2, 2)$

هنا سيحدث فشل (Fail) لأن الرقم الثالث في الاستعلام هو (2)، بينما القاعدة تتوقع أن يكون هو نفسه الرقم الأول. (3)

بما أن المطابقة فشلت قبل الوصول إلى إشارة الـ !، فالمحرك سيعتبر أن القاعدة الأولى لم تتحقق أبداً.

سينتقل المحرك تلقائياً للقاعدة الثانية $\max(X, Y, Y)$ وسيطابقها مع $\max(3, 2, 2)$.

النتيجة ستكون True (صحيح)، وهذا خطأ لأن $\max$ بين 3 و 2 هو 3 وليس 2.

لذلك فإن الحل البرمجي الصحيح هو استخدام متغير ثالث وإجراء التخصيص بعد الـ cut:

$\max(X, Y, Z) :- X \geq Y, !, Z \text{ is } X.$

$\max(X, Y, Z) :- Z \text{ is } Y.$

شرح عمل الكود:

في هذه الحالة، نحن نسأل البرولوج ما هو الماكس بين 3 و 2؟

فيتم الدخول للقاعدة الأولى: ويتم مطابقة $X=3, Y=2$ ويبقى Z متغيراً حراً. (General)

الشرط: يتحقق البرنامج من $3 \geq 2$ النتيجة True

بمجرد عبور الشرط، يصل البرنامج للكت أي وجدت الحل في هذه القاعدة ولن نحاول تجربة القاعدة الثانية أبداً مهما حدث ويتم تنفيذ $Z \text{ is } 3$

وفي حالة $\max(3, 2, 2)$ الاستدعاء سيكون هل الـ 2 هي الماكس بين 3 و 2؟ (والجواب المنطقي هو لا أو False)

1. يتم الدخول للقاعدة الأولى: يتم مطابقة $X=3, Y=2, Z=2$ أي أن Z هنا لم يعد متغيراً حراً بل يحمل قيمة 2
2. الشرط: يتحقق من $3 \geq 2$ النتيجة True
3. يصل للكت ويمنع التراجع هذا يعني أنه مُنع تماماً من الذهاب للقاعدة الثانية.
4. والآن يصل لعملية التخصيص / المساواة $2 \text{ is } 3$
5. وبما أن 2 لا تساوي 3، فإن هذا السطر سيعطي Fail

فرضية العالم المغلق (Closed World Assumption):

وهو ما يعني أن البرنامج يعتبر قاعدة المعرفة المتوفرة لديه هي العالم بأكمله، فكل ما هو موجود فيها يعتبره صحيحاً، وأي معلومة غير مسجلة فيها أو لا يمكن استنتاجها من قواعدها تعتبر خاطئة (False) تلقائياً
فإذا مثلنا قاعدة المعرفة بكتاب متخصص في الحيوانات ولم يذكر هذا الكتاب الفيل ضمن صفحاته لسبب ما فإن البرولوج سيستنتج فوراً أن الفيل ليس حيواناً لأنه يحكم بالفشل على أي معلومة لا يجد لها أصلاً في سجلاته أي أن كل ما هو غير مذكور لديه سيعتبره معلومة خاطئة تماماً

النفي كفشل (Negation as Failure):

يُستخدم الرمز $(\backslash +)$ أو الكلمة المحجوزة (not) للتعبير عن النفي وهنا لا يعني أن الشيء خاطئ في الواقع بل يعني فشل البرولوج في إثبات صحته

مثال الشخص الأعزب (Single): إذا قمنا بتعريف القاعدة كالتالي. $\text{single}(X) :- \backslash + \text{married}(X)$:

أي أن الشخص X أعزب إذا لم نجد معلومة تفيد بأنه متزوج

- إذا سألنا عن علي ($\text{single}(\text{ali})$) ولم يكن اسم علي موجوداً إطلاقاً في قاعدة البيانات كشخص متزوج، سيحاول البرولوج البحث عن $\text{married}(\text{ali})$ ويفشل في إيجادها.
- بسبب هذا الفشل، ستتحقق نتيجة النفي، ويعطي البرنامج إجابة True بأن علي أعزب.
- المشكلة المنطقية: قد يكون علي متزوجاً في الحقيقة، ولكن لعدم وجود المعلومة في قاعدة البيانات، اعتبره البرولوج أعزباً وهذا يوضح أن النفي في البرولوج هو نفي المعرفة وليس نفي الحقيقة المطلقة.

ملاحظة: و يُستخدم النفي بشكل أساسي لضبط العلاقات ومنع النتائج غير المنطقية، مثل علاقة الأخوة أو الأقارب (منع التكرار)

○ مثال علاقة الأخوة (Siblings): عند تعريف الأخوة، قد نعتبر أن أي شخصين لهما نفس الأب هما إخوة ولكن بدون النفي سيعتبر البرنامج أن الشخص هو أخ لنفسه لأن لهما نفس الأب.

○ لحل هذه المشكلة: نستخدم النفي للتأكد من أن المتغيرين ليسا لنفس الشخص بكتابة الشرط

siblings(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.

بهذه الطريقة، يشترط البرنامج أن تكون الهوية مختلفة (not equal) ليتم استكمال استنتاج العلاقة بشكل صحيح

إدارة قاعدة البيانات الديناميكية:

هي القدرة على تعديل الحقائق والقواعد أثناء تشغيل البرنامج (Runtime) مما يسمح للكود بـ التعلم أو تخزين نتائج جديدة لم تكن موجودة عند بدء التشغيل ومنها:

⚡ (Dynamic Predicates)

بشكل افتراضي تكون القواعد والحقائق ثابتة ولكي تتمكن من الإضافة إليها أو الحذف منها أثناء التنفيذ يجب أن نخبر البرنامج مسبقاً بأن هذا Predicates ديناميكي وسنستخدم التعليمية dynamic :- متبوعة باسم Predicates وعدد وسائطه (Arity) والهدف منها السماح للمبرمج بتعديل قاعدة المعرفة برمجياً دون الحاجة لتعديل ملف الكود يدوياً

أوامر التعديل (Assert & Retract):

تستخدم هذه الأوامر للتحكم في محتوى قاعدة المعرفة الحالية:

- assert: يُستخدم لإضافة حقيقة (Fact) أو قاعدة (Rule) جديدة إلى قاعدة البيانات أثناء التنفيذ فمثلاً، إذا لم تكن الحقيقة red(rose) موجودة يمكن إضافتها برمجياً عبر هذا الأمر لتصبح جزءاً من ذاكرة البرنامج
- Retract: يُستخدم لحذف حقيقة أو قاعدة معينة إذا كان هناك أكثر من حقيقة متشابهة، فإنه يحذف أول واحدة يجدها
- retractall: يُستخدم لحذف كافة الحقائق التي تطابق نمطاً معيناً، مثل حذف كل المعلومات المتعلقة Predicates معين مرة واحدة

⚡ التخزين المؤقت (Caching/Memoization):

هذه هي التقنية الأهم لتحسين الأداء حيث تستخدم لتخزين نتائج العمليات الحسابية المعقدة في جدول بحث (Lookup Table) بحيث إذا طُلبت نفس العملية مرة أخرى، يتم جلب النتيجة فوراً من الذاكرة بدلاً من إعادة حسابها

⚡ مثال دالة الجمع والتربيع (Add and Square):

الهدف هو جعل البرنامج يتذكر النتائج التي حسبها سابقاً لكي لا يضطر لإعادة الحساب مرة أخرى

الحل:

add_and_square(X, Y, Z) :- lookup(X, Y, Z), !.

`add_and_square(X, Y, Z) :- Sum is X + Y, Z is Sum * Sum, assert(lookup(X, Y, Z)).`

الشرح:

○ السطر الأول:

أي خبر البرولوج أن يبحث أولاً في الذاكرة فإذا وجد النتيجة يعطها للمستخدم فوراً ويتوقف (باستخدام Cut) واستخدمنا lookup أي انه ديناميكي

○ السطر الثاني:

إذا فشلت القاعدة الأولى (لم يجد النتيجة في الذاكرة) ننتقل لهذه القاعدة التي تقول :
ان نجمع ثم نربع ثم نستخدم assert لنضع النتيجة في الـ lookup حتى يجدها البرنامج في المرة القادمة.

تخيل أننا استدعينا الدالة هكذا

: `add_and_square(3, 4, Z).`

● المرة الأولى (الذاكرة فارغة):

يدخل القاعدة الأولى: يبحث عن `lookup(3, 4, Z)`.. لا يجد شيئاً. (فشل)

ينتقل للقاعدة الثانية:

○ يحسب $3 + 4 = 7$

○ يحسب $7^2 = 49$

`assert(lookup(3, 4, 49))`: النتيجة الان حُزنت.

يخرج بالنتيجة $Z = 49$

● المرة الثانية:

استدعينا نفس الأمر. `add_and_square(3, 4, Z)`:

يدخل القاعدة الأولى: يبحث عن `lookup(3, 4, Z)`.. لأننا سجلناها سابقاً.

بسبب وجود (Cut) ! يعطينا النتيجة 49 ويغلق الملف فوراً



مفهوم وآلية عمل Findall:

في الحالات العادية، عندما نسأل البرولوج عن شيء له نتائج متعددة (مثل: من هم الأشخاص المتزوجون؟)، فإنه يعطينا النتيجة الأولى، ثم ينتظر منا طلب النتيجة التالية عبر التراجع، ليقول $X = \text{أحمد}$ ثم $X = \text{حسام}$ وهكذا. أما دالة Findall، فهي تقوم بجمع كل هذه القيم (أحمد، حسام) وتضعها داخل مصفوفة أو قائمة واحدة، مما يسهل علينا معالجتها لاحقاً ككتلة واحدة، مثل تقسيمها أو عدّ عناصرها

صيغة الدالة (Syntax):

تأخذ الدالة ثلاثة وسائط أساسية:

- المتغير: (X) وهو الشكل أو القيمة التي نريد تجميعها.

- الشرط: (Goal) القاعدة أو الحقيقة التي يجب أن تتحقق (مثل red(X)).

- القائمة: (List) المتغير الذي ستخزن فيه جميع النتائج النهائية.

مثال تجميع العناصر الحمراء: إذا كان لدينا في قاعدة المعرفة عدة حقائق نخبرنا بأن الوردة حمراء والتفاحة حمراء، يمكننا استخدام `findall(X, red(X), L)` هنا سيقوم البرولوج بالبحث عن كل X يحقق شرط أن red ويضعهم جميعاً في القائمة L

○ أو نكتب `findall(red(X), (red(X), beautiful(X)), L)`.

أي أن القائمة لن تكون [rose, flower] بل ستكون [red(rose), red(flower)]. لقد جمعنا النمط كاملاً وليس فقط قيمة المتغير X

مثال:

اكتب تابع يجمع بين فكرة التجميع findall وفكرة split

الحل:

```
number_record(10).
number_record(-5).
number_record(3).

process :- findall(X, number_record(X), L), split(L, Pos, Neg), write('Positive: '),
write(Pos),nl.
write('Negative:'), write(Neg),nl.
split([], [], []).

split([H|T], [H|P], N) :- H >= 0, !, split(T, P, N).
split([H|T], P, [H|N]) :- split(T, P, N).
```

الأسطر من 1 الى 3: تعريف الحقائق

السطر 4 و5: هي القاعدة الأساسية حيث يعمل المحرك الاستدلالي:

○ بمجرد أن ينتهي البرنامج من فحص كل الأرقام رقمًا برقم يقوم بتجميع هذه الاجزاء التي وضعها في القوائم ويعطينا إياها كقائمة كاملة

من السطر 6 للآخر:

تعريف دالة: `split([], [], [])` (شرط التوقف أي ان القائمة فارغة)

`split([H|T], [H|P], N) :- H >= 0, !, split(T, P, N).` وهو سطر الأرقام الموجبة

البرنامج يمسك أول عنصر H إذا كان أكبر من الصفر يضعه فوراً في قائمة الموجب [H|P] ووضعنا ال أي لا تجرب أي قاعدة ثانية لهذا الرقم لأنه حتما موجب و يترك هذا الرقم ويذهب ليفحص باقي القائمة T

سطر الأرقام السالبة: $\text{split}([H|T], P, [H|N]) :- \text{split}(T, P, N).$

البرنامج يصل هنا فقط إذا فشل الشرط في السطر السابق (أي أن الرقم أصغر من صفر) و يضع الرقم (\$H\$) في قائمة السالب [H|N] ثم يذهب ليفحص باقي القائمة T

بما أن الدالة تعمل بنظام الاستدعاء الذاتي (Recursion) سنبدأ بتتبع الاستدعاء على القائمة [10, -5, 3]

■ المرحلة الأولى: فحص الرقم 10

البرنامج يأخذ القائمة [10, -5, 3].

ويضع $H = 10$ ثم يسأل: هل $10 \geq 0$ ؟ نعم.

ضع الـ 10 في قائمة الموجب [10 | P]، ونذهب لفحص باقي القائمة [-5, 3].

■ المرحلة الثانية: نفحص الرقم -5

$H = -5$ ثم يسأل: هل $-5 \geq 0$ ؟ لا.

فينتقل للقاعدة الثانية تلقائياً (قاعدة السالب) ويضع الـ -5 في قائمة السالب [-5 | N]، ويذهب لفحص باقي القائمة [3].

■ المرحلة الثالثة: فحص الرقم 3

$H = 3$ ثم يسأل: هل $3 \geq 0$ ؟ نعم.

فنضع الـ 3 في قائمة الموجب، ونذهب لفحص ما تبقى (وهو قائمة فارغة []).

الآن القائمة فارغة فيذهب للقاعدة الأولى $\text{split}([], [], [])$ وهنا يتوقف البرنامج ويبدأ بتجميع النتائج النهائية. وتكون النتيجة:

○ Positive: [3, 10]

○ Negative: [-5]

ملاحظة:

↪ دالة Setof:

الفرق الجوهرى الذي يميز Setof عن غيرها هو قدرتها على حذف القيم المكررة من القائمة النهائية الناتجة.

• آلية العمل:

عندما يقوم البرولوج بالبحث عن كافة الحلول التي تحقق شرطاً معيناً، قد تظهر نفس النتيجة أكثر من مرة إذا كانت موجودة في قاعدة البيانات بتكرارات مختلفة تقوم Setof تلقائياً بفحص القائمة، وإزالة العناصر المتكررة، لتعطي مجموعة (Set) من العناصر الفريدة فقط

⚡ دالة Bagof:

هي بديل آخر لدالة Findall

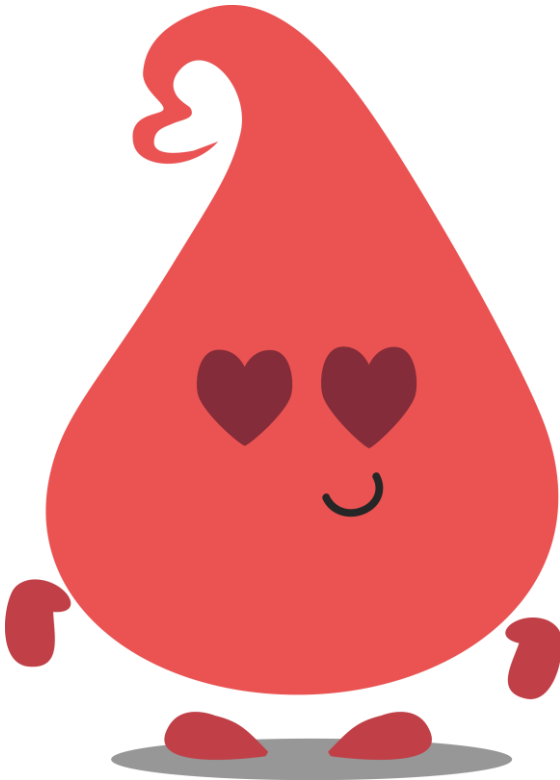
• آلية العمل:

تشبه Findall في أنها تجمع كافة النتائج في قائمة واحدة ولكنها تتميز بكونها تفشل (Fail) إذا لم تجد أي نتائج، عكس Findall التي تعطيك قائمة فارغة وتنجح دائماً.

⚡ متى نستخدم هذه الدوال بدلاً من Findall ؟

الهدف الأساسي من استخدام هذه المجموعة من الدوال (Findall, Bagof, Setof) هو الرغبة في معالجة البيانات ككتلة واحدة.

- على سبيل المثال، إذا أردنا تجميع كافة العناصر التي تحقق شرطاً معيناً ثم تمرير هذه القائمة لدالة أخرى لتقوم بمهام إضافية مثل التقسيم (Split) أو الحساب، فإننا نلجأ لهذه الدوال.
- اختيار Setof تحديداً يكون ضرورياً عندما تكون قاعدة البيانات تحتوي على معلومات مكررة ونريد الحصول على قائمة نظيفة تحتوي على كل حل مرة واحدة فقط.



وإلى هنا تكون قد انتهت محاضرتنا يا أصدقاء
نلتقاكم في المحاضرة القادمة...

انتهت المحاضرة

<The end>